

C Preprocessors

22 July 2026 20:32

The C program is known as 'Source Code'. When the source code is passed through 'preprocessor', it creates 'Expanded Source Code' as per the preprocessor directives used in the source code. Each preprocessor directive begins with a '#' hash symbol. Following preprocessor directives may be used in the source code:

- a. Macro expansion
- b. File inclusion
- c. Conditional compilation
- d. Miscellaneous Directives

Macro Expansion

```
#include<stdio.h>
#define PI 3.1428
int main(int argc, char const *argv[])
{
    float r = 6.25, area;
    area = PI * r * r;
    printf("Area of circle = %f\n", area);
    return 0;
}
```

In the statement:

```
#define PI 3.1428
```

PI is called 'macro template', whereas, 3.1428 is called 'Macro expansion'. During preprocessing every macro template gets replaced with its corresponding macro expansion. Usually macro templates are written in capital letters. This makes it easy for programmers to identify macro template when reading through the program.

And now an important question - why use **#define** at all? Suppose the constant 3.1428 appears many times in your program. Someday you may wish to change all these values with more accurate 3.142857. For this you have to go through the program and manually change each occurrence of the constant. However, if you have defined PI in a **#define** directive, you only need to change the hash defined directive to:

```
#define PI 3.1428
```

Once done, the. Change will be made in all occurrences of Pi during preprocessing. This convenience may not matter for small programs shown above, but with large programs, macro definitions are almost indispensable.

You may feel that same purpose could have been served. Had we used a variable Pi instead of macro template PI. But for three reasons, it would have been a bad idea.

Firstly, it is inefficient since the compiler can generate faster and more compact code for constant than it can for variables. Secondly, using a variable

for what is really a constant encourages sloppy thinking - If something never changes, it is hard to imagine it as a variable. And thirdly, there is always a danger that the variable may inadvertently get altered somewhere in the program. So it is no longer a constant that you think it is.

```
#include<stdio.h>
#define PI 3.1428
#define AND &&
#define OR ||
#define NOT !
#define not_equal !=
#define True 1
#define False 0
```

← It helps us to write the code in a very much in readable form.

```
int main(int argc, char const *argv[])
{
    float r = 6.25, area;
    area = PI * r * r;
    printf("Area of circle = %f\n", area);
    return 0;
}
```

Macro with Arguments

The macros. That we have used so far are simple macros. Macros can have arguments, just as function can. Here is a program that illustrates this fact.

```
#include<stdio.h>
#define PI 3.1428
#define AND &&
#define OR ||
#define NOT !
#define not_equal !=
#define True 1
#define False 0
#define AREA_OF_CIRCLE(r) (PI * r * r)
int main(int argc, char const *argv[])
{
    float r = 6.25, area, r1 = 3.45;
    area = AREA_OF_CIRCLE(r);
    printf("Area of circle = %f\n", area);
    area = AREA_OF_CIRCLE(r1);
    printf("Area of circle = %f\n", area);
    return 0;
}
```

The preprocessor would replace every $AREA_OF_CIRCLE(r)$ with $(PI \times r \times r)$. As it does this, r would be substituted with the argument that we use in the macro. Thus $area = AREA_OF_CIRCLE(r1)$ would be replaced with $area = (PI \times r1 \times r1)$

Output:

```
Area of circle = 122.765625
Area of circle = 37.407177
```

```

#include<stdio.h>
#define PI 3.1428
#define AND &&
#define OR ||
#define NOT !
#define not_equal !=
#define True 1
#define False 0
#define AREA_OF_CIRCLE(r) (PI * r * r)
#define ISDIGIT(y) (y >= '0' AND y <= '9')
#define ISCAPITAL(ch) (ch >= 'A' AND ch <= 'Z')
#define ISSMALL(ch) (ch >= 'a' AND ch <= 'z')

int main(int argc, char const *argv[])
{
    float r = 6.25, area, r1 = 3.45;
    area = AREA_OF_CIRCLE(r);
    printf("Area of circle = %f\n", area);
    area = AREA_OF_CIRCLE(r1);
    printf("Area of circle = %f\n", area);
    char ch;
    printf("Enter any character: ");
    scanf("%c", &ch);
    if(ISDIGIT(ch)){
        printf("The given character is digit.");
    }
    else if(ISCAPITAL(ch)){
        printf("The given charcter is Capital letter.");
    }
    else if(ISSMALL(ch)){
        printf("The given charcter is small letter.");
    }
    else{
        printf("The given character is something else.");
    }
    return 0;
}

```

Output:

```

Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: Q
The given character is Capital letter.
PS C:\Users\bluej\Documents\Students\Sou
Chakraborty\CProgramming\ForGit\Coding\
rams\PreprocessorDirectives> & "C:\Users
j\Documents\Students\Soumodip Chakrabort
ogramming\ForGit\Coding\CPrograms\Prepro
rDirectives\two.exe"
Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: t
The given character is small letter.
PS C:\Users\bluej\Documents\Students\Sou
Chakraborty\CProgramming\ForGit\Coding\
rams\PreprocessorDirectives> & "C:\Users
j\Documents\Students\Soumodip Chakrabort
ogramming\ForGit\Coding\CPrograms\Prepro
rDirectives\two.exe"
Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: 9
The given character is digit.
PS C:\Users\bluej\Documents\Students\Sou
Chakraborty\CProgramming\ForGit\Coding\
rams\PreprocessorDirectives> & "C:\Users
j\Documents\Students\Soumodip Chakrabort
ogramming\ForGit\Coding\CPrograms\Prepro
rDirectives\two.exe"
Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: ^
The given character is something else.

```

Here are some important points to remember while writing macros with arguments.

- Do not give a space between macro template and its argument in **#define**.
- The entire macro expansion should be enclosed within parenthesis.
- Macros can be split into multiple lines with a ('\')back slash present at the end of each line except the last. For example:

```

#include<stdio.h>
#define PI 3.1428
#define AND &&
#define OR ||
#define NOT !
#define not_equal !=
#define True 1
#define False 0
#define AREA_OF_CIRCLE(r) (PI * r * r)
#define ISDIGIT(y) (y >= '0' AND y <= '9')
#define ISCAPITAL(ch) (ch >= 'A' AND ch <= 'Z')
#define ISSMALL(ch) (ch >= 'a' AND ch <= 'z')
#define HLINE(start, end) \
do{\
    int i = (start);\
    for(; i < end; i++)\
        printf("%c", 196);\
}while(0);

```

```

int main(int argc, char const *argv[])
{
    float r = 6.25, area, r1 = 3.45;
    area = AREA_OF_CIRCLE(r);
    printf("\nArea of circle = %f\n", area);
    HLINE(1, 30);
    area = AREA_OF_CIRCLE(r1);
    printf("\nArea of circle = %f\n", area);
    HLINE(1,30);
    char ch;
    printf("\nEnter any character: ");
    scanf("%c", &ch);
    HLINE(1,30);
    if(ISDIGIT(ch)){
        printf("\nThe given character is digit.\n");
    }
    else if(ISCAPITAL(ch)){
        printf("\nThe given charcter is Capital letter.\n");
    }
    else if(ISSMALL(ch)){
        printf("\nThe given charcter is small letter.\n");
    }
    else{
        printf("\nThe given character is something else.\n");
    }
    HLINE(1,30);
    return 0;
}

```

Output:

```

Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: 6
The given character is digit.

```

- d. If you are unable to debug a macro, you should view the expanded code of the program to see how the macro is getting expanded. If your source code is present in file two.c, then the expanded source code would be stored in two.I . You need to generate this file at the command prompt by saying:

```
gcc -E two.c -o two.i
```

This command invoked the C preprocessor, which generates the expanded source code and stores it in the file called two.I, You can now open this file and see the expanded source code.

Macros versus Functions

In the above example, a macro AREA was used to calculate the area of the circle. We could have written a function area () for the same. This brings us

to a question, when one to use when?

If we use a macro, it would be expanded during preprocessing. As against this, if we use a function during execution control and the value of radius would be passed to area. Area would be calculated and returned back.

So if we use macro 100 times in a program, the macro expansion (formula) goes into our source code at 100 different places, thus increasing the program size. On the other hand, if a function is used, the formula would occur only once in the function. The hundred places where you need this formula, there would be a function calls. So space requirement would now be less. But passing arguments to a function and getting back the returned value does take time and would therefore slow down the program. This gets avoided with macros since they have already been expanded and placed in the source code before compilation. Does the trade-off between memory space and time.

More or less, if the macro is simple as in our examples, it makes nice short hands and avoid the overheads associated with function calls. On the other hand, if we have a fairly large macro and it is used fairly often, we ought to replace it with our function.

File Inclusion

The next preprocessor directive that we will explore is file inclusion. It looks like this:

```
#include "filename"
```

It causes the entire content of filename to be inserted into the source code where we have used #include. It is common for the files that are to be included to have .h extension. This extension stands for "header file" as its content when included, go to the head of your program.

The prototypes of all the library functions are grouped into different categories and then stored in different header files. For example, prototypes of all math related functions are stored in a file 'math.h', prototypes of input/output functions are stored in the file stdio.h etc.

Actually there exist two ways to write #include statement. These are:

1) #include "filename"

2) #include <filename>

The meaning of each of these forms is given below:

#include "mylib.h"	This command would look for the file mylib.h in the current directory as well as specified list of directories as mentioned in the include search path that might have been set up.
#include<mylib.h>	This command would look for the file mylab.h in the specified list of directories only.

The include search path is a list of directories that would be searched for the file being included. Different C compilers let you set the search path in different manner.

Conditional Compilation

We can, if we want, have the compiler skip over part of a source code by inserting the preprocessor command `#ifdef` and `#endif`, which have the general form given below.

```
#ifdef macroname
Statement 1;
Statement 2;
Statement 3;
#endif
```

If macro name has been `#defined`, the block of code will be processed as usual, otherwise not.

Where would `#ifdef` be useful? When would you like to compile only a part of your program? In three different cases:

- i. To "comment out" some lines of code that we do not need right now as shown below.

```
int main(){
    #ifdef NOTNOW
        Statement 1;
        Statement 2;
    #endif
    Statement 3;
    Statement 4;
}
```

Here statements one and two would get compiled only if the macro `NOTNOW` has been defined, and we have purposefully omitted the definition of this macro. At a later date if we want that these statements should also get compiled, we can either delete the `#ifdef` and `#endif` statements or `#define NOTNOW` at the top.

- ii. A more sophisticated use of `#ifdef` has to do with making the programs portable, that is to make them work on two computers with different configurations. You can do so by isolating the lines of code that must be different for each machine by making them off with `#ifdef` as follows:

```
int main(){
    #ifdef INTEL
        Code suitable for Intel PC
    # else
        Code suitable for Arm PC (Motorola, Apple, Snapdragon, Mediatek)
    #endif
}
```

So when we compile this program, it would compile only the codes suitable for Motorola PC and the common code since macro `INTEL` has not been defined. If we want to run the program on Intel PC before recompiling the program, we need to add the statement at the top saying,

```
#define INTEL
```

Sometimes, instead of `#ifdef`, the `#ifndef` directive is used. The `#ifndef` (Which means 'if not defined') works exactly opposite to `#ifdef`.

- iii. Ideally we should `#include` a file only once. But if by mistake we end up including it twice, it should get included only once. This can be achieved using `#ifndef` as shown below:

```
/*myfile.h*/
#ifndef __myfile_h
#define __myfile_h
/*some declaration*/
#endif
```

First time the file 'myfile.h' gets included the preprocessor checks whether a macro called `__myfile_h` has defined or not. If it has not been, then it gets defined and the rest of the code gets included. Next time we attempt to include the same file, the inclusion is prevented since `__myfile_H` already stands defined.

#if and #elif Directives

The `#if` directive can be used to test whether an expression evaluates to non-zero value or not. If the result of the expression is non - zero then subsequent lines up to `#else`, `#elif` or `#endif` are compiled, otherwise they are skipped. A simple example of `#if` directive is shown below:

```
int main(){
    #if TEST <= 5
        Statement 1;
    #else
        Statement 2;
    #endif
}
```

If the expression `TEST <= 5` Evaluates to true, then statement one is compiled. Otherwise statement 2 is compiled. In place of expression `TEST <= 5`, other expression like `(LEVEL == HIGH || LEVEL == LOW)` or `ADAPTER == SVGA` can also be used. If required, we can even use nested conditional compilation directives.

The build Process

There are many steps involved in converting a **C program** into an executable form. Figure below shows these different steps along with the files created during each stage. Many software development tools hide some of these from us. However, if you understand these steps, it will make you a better programmer.



